



FastAPI MASTERY SERIES

FastAPI

The Complete Hands-On Guide

From single-file scripts to a real, structured, database-backed, authenticated API ready for a team to build on.

PHASE 2 · INTERMEDIATE

Dependency Injection

Routers & Project Structure

Async / Await

SQLAlchemy (sync)

Async SQLAlchemy

Alembic Migrations

OAuth2 + JWT Auth

Middleware & CORS

Custom Error Handling

Background Tasks

Hands-on project for this phase: **The TaskFlow API** · Built on **FastAPI 0.132+**, **Python 3.10+**, **SQLAlchemy 2.0**, **PyJWT**, **pwdlib**

Verified against the official documentation at fastapi.tiangolo.com · 2026 Edition

From Phase 1 to Phase 2

Phase 1 taught you the request → function → response loop, type-driven validation, and the automatic docs. Phase 2 is where toy examples grow up: a real database, real users with passwords, a project layout that scales, and the safety nets professional APIs depend on.

What you should already be comfortable with

- ✓ Creating a FastAPI app and running it with `fastapi dev main.py`
- ✓ Path operations for GET / POST / PUT / DELETE, with path and query parameters
- ✓ Pydantic models, including separate input and output models with validation
- ✓ Response models, status codes, and `HTTPException` for errors

If any of those feel hazy, revisit Phase 1 first — Phase 2 builds directly on them and never re-explains them from scratch.

YOUR PHASE 2 PROJECT



The TaskFlow API

A multi-user task manager. Each user signs up, logs in, and manages their own tasks. By the end of this phase TaskFlow will have a real SQL database, schema migrations, JWT-protected endpoints, CORS for a future frontend, structured error handling, and background work for things like sending confirmation emails. Every chapter adds one production-grade capability.

The mental shift Phase 2 asks of you

Phase 1 lived inside one file. Phase 2 introduces three new habits that change how you think:

1. **Compose, do not repeat.** Dependency Injection lets you write each rule once and inject it wherever it is needed.
2. **Persist properly.** An in-memory list disappears on every reload — real data lives in a database and evolves with migrations.
3. **Defend the perimeter.** Real users mean authentication, real frontends mean CORS, real failures mean coherent error responses.

All code follows the modern FastAPI style used in the current official documentation: `Annotated` dependencies, SQLAlchemy 2.0 syntax, `PyJWT` for tokens, and lifespan events instead of legacy startup/shutdown.

What you will learn

1 Dependency Injection (Depends)

The composition pattern that powers everything else

2 Routers & Project Structure (APIRouter)

From one file to a proper, scalable layout

3 Async / Await in FastAPI

When to write `def`, when to write `async def`, and why

4 Databases with SQLAlchemy (sync)

Models, sessions, CRUD against a real database

5 Async Databases (async SQLAlchemy)

`AsyncSession`, `await`, and async drivers

6 Alembic Migrations

Evolving the database schema safely as your code changes

7 Authentication — OAuth2 & JWT

Users, passwords, login, and protected endpoints

8 Middleware (CORS & custom)

Cross-cutting logic on every request and response

9 Error Handling & Custom Exception Handlers

Clean, consistent failure responses

10 Background Tasks

Doing work after the response is sent

★ Capstone — The Complete TaskFlow API

All ten capabilities assembled into a single, runnable application

§ Glossary of Intermediate Terms

→ What Comes Next — Phase 3 Preview

Dependency Injection (Depends)

Dependency Injection is FastAPI's superpower. It lets you declare what each endpoint *needs* — pagination, a database session, the logged-in user — and FastAPI provides it. The rest of Phase 2 (databases, auth, transactions) leans on this single idea.

The problem it solves

Look at three endpoints that all need pagination:

```
@app.get("/tasks")
def list_tasks(skip: int = 0, limit: int = 10):
    return tasks[skip : skip + limit]

@app.get("/users/{user_id}/tasks")
def user_tasks(user_id: int, skip: int = 0, limit: int = 10):
    ...

@app.get("/tasks/archived")
def archived(skip: int = 0, limit: int = 10):
    ...
```

The same parameters are copied three times. Today it is harmless; the day you change the limit cap from 10 to 100 you must change three files. The day you add a `sort_by` parameter you update everywhere. This is exactly the duplication DI eliminates.

The idea

A API TERM — Dependency Injection

A pattern where a function declares what it needs (its *dependencies*) and someone else supplies them. In FastAPI you declare a dependency by writing a function, and you inject it with `Depends()`. FastAPI calls the dependency, gives the result to your endpoint, and caches it within the request.

Concretely: write the shared logic once as a function, then declare endpoints "depend on" it.

```
from typing import Annotated
from fastapi import Depends, FastAPI, Query

app = FastAPI()

def pagination(
    skip: int = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 10,
) -> dict:
    return {"skip": skip, "limit": limit}
```

```
PaginationDep = Annotated[dict, Depends(pagination)]

@app.get("/tasks")
def list_tasks(page: PaginationDep):
    return tasks[page["skip"] : page["skip"] + page["limit"]]

@app.get("/tasks/archived")
def archived(page: PaginationDep):
    return archived_tasks[page["skip"] : page["skip"] + page["limit"]]
```

Reading the syntax

- `pagination` is just a regular function whose signature defines query parameters (Phase 1 rules still apply).
- `Depends(pagination)` says "call this function and inject its result."
- `PaginationDep = Annotated[dict, Depends(pagination)]` is the modern style: define the dependency type once, then reuse it. The official docs use exactly this pattern to keep endpoint signatures short.
- Inside the endpoint, `page` is just a dictionary — nothing magical.

? WHY IT MATTERS — One source of truth

Add a `sort_by` field to the `pagination` function and every endpoint that depends on it gains the parameter — with docs, validation, and defaults — for free. This is what we mean by 'compose, do not repeat.'

What can be a dependency?

Any callable: a function, an `async def` function, a class, even another dependency. Common kinds in real apps:

Dependency	Used for	We will use it in
A small function returning pagination/filter info	DRY parameter sets	This chapter
A function yielding a database session	Per-request DB access	Chapter 4
A function decoding the JWT token	The current logged-in user	Chapter 7
A class with attributes for config	Settings / API keys	Throughout

Classes as dependencies

If your dependency needs structure, write it as a class. The class's `__init__` becomes the dependency signature; FastAPI gives you an instance.

```

class TaskFilters:
    def __init__(
        self,
        status: str | None = None,
        owner_id: int | None = None,
        skip: int = 0,
        limit: Annotated[int, Query(le=100)] = 10,
    ):
        self.status = status
        self.owner_id = owner_id
        self.skip = skip
        self.limit = limit

@app.get("/tasks")
def list_tasks(filters: Annotated[TaskFilters, Depends()]):
    return apply_filters(tasks, filters)[filters.skip : filters.skip + filters.limit]

```

★ PRO TIP — Depends() with no argument

When the type is itself the dependency (a class), you can write `Depends()` with empty parentheses. FastAPI infers the callable from the type. Clean, idiomatic.

Sub-dependencies (dependencies of dependencies)

Dependencies can depend on other dependencies. FastAPI resolves the whole chain for you and — importantly — calls each one **once per request**, caching the result.

```

def get_settings() -> Settings: ...
def get_db(settings: Annotated[Settings, Depends(get_settings)]): ...
def get_current_user(db: Annotated[Session, Depends(get_db)], token: str = ...): ...

@app.get("/tasks/mine")
def my_tasks(user: Annotated[User, Depends(get_current_user)]):
    return user.tasks

```

One request, one settings object, one DB session, one current-user lookup — even though three endpoints might transitively use them. This is how Phase 2's later chapters build the auth flow.

Dependencies with yield: setup & teardown

Some resources need cleanup after the request — a database session must be closed, a file handle released. `yield` turns a dependency into a setup-then-teardown generator.

```

def get_db():
    db = SessionLocal()           # setup, before the request
    try:
        yield db                  # this is what gets injected
    finally:
        db.close()                # teardown, after the response is sent

```

A API TERM — Yield dependency

A dependency that `yield` s a value instead of returning. Code before `yield` runs before your endpoint; code after runs once the response is finished. This is the standard FastAPI pattern for opening and reliably closing resources.

Where dependencies can be attached

Three useful scopes, from narrowest to broadest:

Scope	How	Use case
One endpoint	Parameter with <code>Depends()</code>	Pagination, current user
A whole router	<code>APIRouter(dependencies=[...])</code>	Require login for all task routes
Whole app	<code>FastAPI(dependencies=[...])</code>	Global rate limit, request ID

▶ TRY IT YOURSELF — Add pagination DI to TaskFlow

Create a fresh `main.py` with a `tasks` list, build the `pagination` dependency, and use it on a `GET /tasks` endpoint. Confirm the docs at `/docs` show the `skip` and `limit` parameters as if you had declared them directly.

Routers & Project Structure

One `main.py` is great until it isn't. Around 200 lines, the file becomes a maze. `APIRouter` lets you slice your API into focused modules, and a sensible folder layout keeps a growing project navigable.

APIRouter: a mini-FastAPI per feature

A API TERM — APIRouter

A lightweight grouping object that exposes the same decorator methods as `FastAPI()` — `.get`, `.post`, and so on. You define endpoints on it, then attach the whole router to the main app with `app.include_router(...)`. Think of it as 'a feature in a box.'

app/routers/tasks.py

```
# app/routers/tasks.py
from fastapi import APIRouter

router = APIRouter(prefix="/tasks", tags=["Tasks"])

@router.get("")
def list_tasks(): ...

@router.get("/{task_id}")
def get_task(task_id: int): ...
```

Notice the `prefix="/tasks"`: every route in this file is automatically served under `/tasks`, so the decorator paths stay short. The `tags` argument groups these endpoints together in the docs.

Wiring routers into the app

app/main.py

```
# app/main.py
from fastapi import FastAPI
from app.routers import tasks, users, auth

app = FastAPI(title="TaskFlow API")

app.include_router(auth.router)
app.include_router(users.router)
app.include_router(tasks.router)
```

Each router stays focused; the main file simply assembles them. Adding a new feature is now a matter of creating one file under `routers/` and one `include_router` line.

Router-level dependencies and prefixes

You can attach dependencies and tags to the whole router at once. This is how, in Chapter 7, we will require authentication for every task endpoint with a single line.

```
router = APIRouter(
    prefix="/tasks",
    tags=["Tasks"],
    dependencies=[Depends(get_current_user)], # every task route is protected
    responses={401: {"description": "Not authenticated"}},
)
```

A scalable project layout

There is no one mandated layout, but the structure below is the one used by the official *full-stack-fastapi-template* and most production projects. It is what we will adopt for the rest of Phase 2.

Project layout

```
taskflow/
├─ app/
│   ├── __init__.py
│   ├── main.py           # FastAPI() and include_router calls
│   ├── core/
│   │   ├── config.py     # Settings (env vars, secret keys)
│   │   └─ security.py    # Hashing & JWT helpers (Ch. 7)
│   ├── db/
│   │   ├── base.py       # Declarative base class
│   │   └─ session.py     # Engine + get_db dependency (Ch. 4)
│   ├── models/           # SQLAlchemy ORM models
│   │   ├── user.py
│   │   └─ task.py
│   ├── schemas/          # Pydantic models (request/response)
│   │   ├── user.py
│   │   └─ task.py
│   ├── routers/          # APIRouters, one per feature
│   │   ├── auth.py
│   │   ├── users.py
│   │   └─ tasks.py
│   ├── crud/             # Pure DB functions, no HTTP knowledge
│   │   ├── user.py
│   │   └─ task.py
│   └─ dependencies.py    # Shared Depends() callables
├─ alembic/               # Migrations (Ch. 6)
├─ .env
├─ alembic.ini
└─ pyproject.toml
```

The role of each layer

Folder	Knows about	Does not know about
routers/	HTTP, request shapes, status codes	SQL
crud/	The database (sessions, queries)	HTTP
models/	Table definitions only	Anything else
schemas/	Pydantic shapes for API in/out	The database

? WHY IT MATTERS — Separation of concerns pays off fast

When the database changes, you touch `models/` and `crud/`. When the API shape changes, you touch `schemas/` and `routers/`. Tests, refactoring, and team work all become easier because the layers cannot reach into each other.

Avoiding circular imports

The single most common growing-pain when you split files: `main.py` imports `tasks.py`, which imports something that imports `main.py`. The fix is the rule of *flow*: low-level modules (`core/`, `db/`, `models/`) never import from high-level ones (`routers/`, `main.py`). Imports flow upward only.

★ PRO TIP — Use absolute imports

Write `from app.crud import task as crud_task`, not `from ..crud import task`. Absolute imports are easier to read, less fragile to file moves, and the convention used by the official FastAPI template.

▶ TRY IT YOURSELF — Refactor TaskFlow into the layout above

Create the folders, move `list-tasks/get-task` into `app/routers/tasks.py`, and wire it up in `app/main.py`. Run with `fastapi dev app/main.py`. The docs should look identical — that is the point. Structure is for humans, not the runtime.

Async / Await in FastAPI

FastAPI is asynchronous at its core. You can use that power without ever learning it — every endpoint you wrote in Phase 1 was a regular `def` function — but understanding async is what unlocks the framework's full speed and lets you use modern async database drivers (next chapter).

The problem async solves

A web request usually spends most of its time *waiting*: waiting for a database to respond, waiting for a third-party API, waiting for a file to be read. A traditional synchronous server, while waiting on one request, sits idle and cannot serve others.

A API TERM — Synchronous vs asynchronous

Synchronous code runs top-to-bottom, blocking on each step. **Asynchronous** code can pause at well-defined points (an `await`) and let the runtime do other work in the meantime, then resume when the awaited operation finishes. The result: one process handles many concurrent requests on a single thread.

The two keywords

Keyword	Where it goes	What it means
<code>async def</code>	On the function definition	This function may pause and resume
<code>await</code>	Before a call inside an <code>async def</code>	Pause until that thing finishes

```
import asyncio

async def fetch_user(user_id: int):
    await asyncio.sleep(0.1)  # placeholder for a real I/O call
    return {"id": user_id, "name": "Ana"}

@app.get("/users/{user_id}")
async def read_user(user_id: int):
    return await fetch_user(user_id)
```

The single rule: **you can only `await` inside an `async def`**. Everything else flows from that.

The golden rule: `async def` vs `def`

This is the only practical decision you have to make per endpoint. FastAPI supports both, and it does the right thing automatically — but you must pick correctly or you will accidentally block the entire server.

What your endpoint does inside	Declare it as	Why
Calls async libraries (<code>await db.execute(...)</code>)	<code>async def</code>	Required — you cannot await otherwise
Calls sync, blocking libraries (sync SQLAlchemy, requests)	<code>def</code>	FastAPI runs it in a threadpool
Pure CPU work / no I/O at all	Either works	Prefer <code>def</code> for clarity

? WHY IT MATTERS — Sync code in async def is the classic blocker

If you declare an endpoint `async def` and then call a *blocking* library (like the sync `requests` package or sync SQLAlchemy), you freeze the entire server while that call runs — no other requests can be served. The safe rule: only mix in async libraries inside `async def`, or just use plain `def` and let FastAPI handle it.

Dependencies can be async too

A dependency is just a callable, so the same rules apply. The current user lookup we will build in Chapter 7 will be `async` because token validation is I/O-light but the database session it uses is async.

```
async def get_current_user(token: str, db: AsyncSession) -> User:
    payload = jwt.decode(token, SECRET, algorithms=["HS256"])
    user = await db.get(User, payload["sub"])
    if user is None:
        raise HTTPException(401)
    return user
```

Running blocking work without blocking

Occasionally you must call a slow synchronous function from inside an `async def`. Use `run_in_threadpool` to push it onto a worker thread — the event loop stays free.

```
from fastapi.concurrency import run_in_threadpool

@app.get("/report")
async def report():
    data = await run_in_threadpool(generate_pdf, params)
    return {"ready": True}
```

What about CPU-bound work?

Async does not help when the work is *compute*, not I/O — a number-crunching loop will still hog the event loop. For genuinely heavy CPU tasks (image processing, machine learning), offload to a task queue. We cover that in Phase 3 with Celery; for short one-off post-response work, Phase 2 has Background Tasks (Chapter 10).

★ **PRO TIP — Pragmatic default**

Until you specifically need an async library, just write `def`. FastAPI's threadpool handles it. When you need the async database driver of Chapter 5, switch those endpoints to `async def` and let the change propagate from there.

▶ **TRY IT YOURSELF — Feel the difference**

Make two endpoints: one with `def` + `time.sleep(2)`, another with `async def` + `await asyncio.sleep(2)`. Open four browser tabs hitting each in parallel. Notice how the async one finishes all four in roughly 2 seconds while the sync one takes 8 — FastAPI's threadpool size determines how many sync calls can overlap.

Databases with SQLAlchemy (sync)

In-memory lists vanish on reload. A real API stores data in a database. We will use **SQLAlchemy 2.0**, the standard Python ORM, in its synchronous form first because it is conceptually simplest. The next chapter converts it to async.

The pieces, in plain words

A API TERM — ORM, engine, session, model

An **ORM** (Object-Relational Mapper) lets you work with database rows as Python objects instead of writing raw SQL. The **engine** manages the actual connection to the database. A **session** is your conversation with the database for one unit of work — opened, used, then closed. A **model** is a Python class mapped to a table.

Layer	Lives in	Owns
Engine	<code>app/db/session.py</code>	Connection pool to the database
Session	<code>get_db</code> dependency	One transaction, one request
Models	<code>app/models/</code>	Table definitions (User, Task)
Schemas	<code>app/schemas/</code>	Pydantic shapes — separate from models

Install the packages

```
pip install sqlalchemy
# For PostgreSQL: pip install psycopg2-binary
# For SQLite, nothing else — it ships with Python
```

The engine and session factory

app/db/session.py

```
# app/db/session.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session
from typing import Annotated
from fastapi import Depends

# For SQLite (file lives next to the app); use the proper URL for Postgres.
DATABASE_URL = "sqlite:///./taskflow.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}, # required for SQLite + FastAPI
```

```

)

SessionLocal = sessionmaker(
    autocommit=False, autoflush=False, bind=engine,
)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

SessionDep = Annotated[Session, Depends(get_db)]

```

That last line, `SessionDep = Annotated[Session, Depends(get_db)]`, is the line you will write into every endpoint. It is the entire glue between the database and your routes.

i NOTE — Why `check_same_thread=False`

SQLite normally forbids using one connection from multiple threads. FastAPI's threadpool uses multiple threads, so we have to opt out. PostgreSQL and other real databases need no such flag.

Modern SQLAlchemy 2.0 models

SQLAlchemy 2.0 introduced a type-hint-friendly declarative style. It pairs beautifully with FastAPI's existing love of type hints.

app/db/base.py

```

# app/db/base.py
from sqlalchemy import DeclarativeBase

class Base(DeclarativeBase):
    pass

```

app/models/task.py

```

# app/models/task.py
from datetime import datetime
from sqlalchemy import String, DateTime, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.db.base import Base

class Task(Base):
    __tablename__ = "tasks"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(120))
    description: Mapped[str | None] = mapped_column(String(500), default=None)

```

```

is_done: Mapped[bool] = mapped_column(default=False)
created_at: Mapped[datetime] = mapped_column(
    DateTime, default=datetime.utcnow,
)
owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"))

```

A API TERM — Mapped & mapped_column

The 2.0-style way to declare a column: the type hint (`Mapped[str]`) tells you the Python type, and `mapped_column` passes SQL details (length, foreign keys, defaults). Editor autocompletion and type-checkers understand it natively — a major upgrade over the old style.

Pydantic schemas vs SQLAlchemy models

This is the most common confusion. The two look similar but serve different worlds:

SQLAlchemy model	Pydantic schema
Lives in <code>app/models/</code>	Lives in <code>app/schemas/</code>
Maps to a database table	Validates HTTP input / shapes output
Instances are loaded from the DB	Instances come from JSON bodies
Used by <code>crud/</code>	Used by <code>routers/</code>

`app/schemas/task.py`

```

# app/schemas/task.py
from datetime import datetime
from pydantic import BaseModel, Field

class TaskCreate(BaseModel):
    title: str = Field(min_length=1, max_length=120)
    description: str | None = Field(default=None, max_length=500)

class TaskPublic(BaseModel):
    id: int
    title: str
    description: str | None
    is_done: bool
    created_at: datetime

model_config = {"from_attributes": True}  # allow ORM -> Pydantic

```

A API TERM — from_attributes

By default Pydantic builds models from dicts. `from_attributes=True` lets it build from objects with attributes — exactly what SQLAlchemy returns. With this flag, FastAPI's `response_model` can convert ORM rows into JSON automatically. (In Pydantic v1 this was called `orm_mode`.)

CRUD: pure database functions

Keep database code out of your routes. The `crud/` layer takes a session and primitive arguments, performs the work, and returns ORM objects.

`app/crud/task.py`

```
# app/crud/task.py
from sqlalchemy import select
from sqlalchemy.orm import Session
from app.models.task import Task
from app.schemas.task import TaskCreate

def create(db: Session, data: TaskCreate, owner_id: int) -> Task:
    task = Task(**data.model_dump(), owner_id=owner_id)
    db.add(task)
    db.commit()
    db.refresh(task)
    return task

def get(db: Session, task_id: int) -> Task | None:
    return db.get(Task, task_id)

def list_for_owner(db: Session, owner_id: int, skip: int, limit: int) -> list[Task]:
    stmt = (
        select(Task)
        .where(Task.owner_id == owner_id)
        .offset(skip).limit(limit)
        .order_by(Task.created_at.desc())
    )
    return list(db.scalars(stmt))

def delete(db: Session, task: Task) -> None:
    db.delete(task)
    db.commit()
```

Wiring it into a router

`app/routers/tasks.py`

```
# app/routers/tasks.py
from fastapi import APIRouter, HTTPException, status
from app.db.session import SessionDep
from app.schemas.task import TaskCreate, TaskPublic
from app.crud import task as crud_task

router = APIRouter(prefix="/tasks", tags=["Tasks"])

@router.post("", response_model=TaskPublic, status_code=status.HTTP_201_CREATED)
```

```
def create_task(data: TaskCreate, db: SessionDep):
    return crud_task.create(db, data, owner_id=1)  # owner from auth in Ch. 7

@router.get("/{task_id}", response_model=TaskPublic)
def get_task(task_id: int, db: SessionDep):
    task = crud_task.get(db, task_id)
    if task is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Task not found")
    return task
```

Look at the cleanliness: the route knows about HTTP, the CRUD function knows about the database, and Pydantic + `from_attributes=True` converts the ORM object to JSON. Each layer does one thing.

Creating the tables

For a quick start you can ask SQLAlchemy to create tables from your models on app startup — fine for development, but in Chapter 6 we replace this with proper migrations.

app/main.py

```
# app/main.py — temporary, replace with Alembic in Ch. 6
from contextlib import asynccontextmanager
from fastapi import FastAPI
from app.db.base import Base
from app.db.session import engine
from app.routers import tasks

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)  # create tables on startup
    yield

app = FastAPI(title="TaskFlow API", lifespan=lifespan)
app.include_router(tasks.router)
```

API TERM — Lifespan

A function decorated with `@asynccontextmanager` that runs setup code before `yield` and shutdown code after. Pass it as `lifespan=` to `FastAPI()`. This is the modern replacement for the older `@app.on_event("startup")` decorators, which are now deprecated. We cover lifespans in depth in Phase 3.

TRY IT YOURSELF — Persist a task

Set this all up, run `fastapi dev app/main.py`, then `POST /tasks` with a JSON body in `/docs`. Stop the server, restart it, and `GET /tasks/1`. Your task survived — you have a real database-backed API.

Async Databases (async SQLAlchemy)

Sync SQLAlchemy works fine — FastAPI runs your `def` endpoints in a threadpool. But going async unlocks higher concurrency under load and removes a whole class of "I forgot which endpoint was async" decisions. Same API, slightly different syntax.

What changes, conceptually

Synchronous	Asynchronous
<code>create_engine</code>	<code>create_async_engine</code>
<code>Session</code> / <code>sessionmaker</code>	<code>AsyncSession</code> / <code>async_sessionmaker</code>
<code>db.execute(...)</code>	<code>await db.execute(...)</code>
<code>def endpoint(...)</code>	<code>async def endpoint(...)</code>
Sync driver (<code>psycopg2</code>)	Async driver (<code>asyncpg</code> , <code>aiosqlite</code>)

The models themselves do *not* change — your `Task` and `User` classes are identical. Only the engine, session, and call style differ.

Install an async driver

```
pip install "sqlalchemy[asyncio]"
# Drivers:
pip install aiosqlite          # for SQLite
pip install asyncpg            # for PostgreSQL
```

Async engine and session

app/db/session.py

```
# app/db/session.py (async version)
from sqlalchemy.ext.asyncio import (
    create_async_engine, AsyncSession, async_sessionmaker,
)
from typing import Annotated
from fastapi import Depends

DATABASE_URL = "sqlite+aiosqlite:///./taskflow.db"
# PostgreSQL: "postgresql+asyncpg://user:pass@host/dbname"

engine = create_async_engine(DATABASE_URL)
```

```
AsyncSessionLocal = async_sessionmaker(
    engine, expire_on_commit=False, class_=AsyncSession,
)
```

```
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session
```

```
SessionDep = Annotated[AsyncSession, Depends(get_db)]
```

A API TERM — `expire_on_commit=False`

By default, SQLAlchemy expires object attributes after each commit so any later access re-fetches from the database. With async code that turns innocent attribute reads into unexpected `await` s. Setting `expire_on_commit=False` is the standard pattern in async FastAPI projects.

Async CRUD

The patterns mirror the sync version one-for-one:

app/crud/task.py – async

```
# app/crud/task.py (async version)
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from app.models.task import Task
from app.schemas.task import TaskCreate

async def create(db: AsyncSession, data: TaskCreate, owner_id: int) -> Task:
    task = Task(**data.model_dump(), owner_id=owner_id)
    db.add(task)
    await db.commit()
    await db.refresh(task)
    return task

async def get(db: AsyncSession, task_id: int) -> Task | None:
    return await db.get(Task, task_id)

async def list_for_owner(
    db: AsyncSession, owner_id: int, skip: int, limit: int,
) -> list[Task]:
    stmt = (
        select(Task)
        .where(Task.owner_id == owner_id)
        .offset(skip).limit(limit)
        .order_by(Task.created_at.desc())
    )
```

```
result = await db.scalars(stmt)
return list(result)
```

Async routers

The endpoints become `async def` and `await` the CRUD functions. Everything else — the response model, the status code, the `HTTPException` — is identical to the sync version.

app/routers/tasks.py – async

```
# app/routers/tasks.py
from fastapi import APIRouter, HTTPException, status
from app.db.session import SessionDep
from app.schemas.task import TaskCreate, TaskPublic
from app.crud import task as crud_task

router = APIRouter(prefix="/tasks", tags=["Tasks"])

@router.post("", response_model=TaskPublic, status_code=status.HTTP_201_CREATED)
async def create_task(data: TaskCreate, db: SessionDep):
    return await crud_task.create(db, data, owner_id=1)

@router.get("/{task_id}", response_model=TaskPublic)
async def get_task(task_id: int, db: SessionDep):
    task = await crud_task.get(db, task_id)
    if task is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Task not found")
    return task
```

Loading relationships safely

One async gotcha worth knowing: lazy-loading a relationship triggers a database call, and async drivers reject implicit I/O outside an `await`. The fix is to load related data *eagerly* as part of the query.

```
from sqlalchemy.orm import selectinload

stmt = (
    select(User)
    .options(selectinload(User.tasks))  # load tasks in one extra SELECT
    .where(User.id == user_id)
)
user = (await db.scalars(stmt)).one()
for task in user.tasks:  # safe: already loaded
    print(task.title)
```

! WATCH OUT — MissingGreenlet / lazy loading errors

If you see `MissingGreenlet` or 'cannot operate on closed transaction' errors, you almost certainly accessed a relationship that wasn't loaded. Add `selectinload` (or `joinedload`) to the query and the error disappears.

Sync or async — how to choose?

Choose sync if...	Choose async if...
You're learning, or your team mostly knows sync	You want maximum concurrent throughput
Your DB driver is sync-only	You can use <code>asyncpg</code> / <code>aiomysql</code> / <code>aiosqlite</code>
You mix in many other sync libraries	Your stack (Redis, HTTP clients) is already async
Code clarity beats raw speed for your case	You'll later add WebSockets or Server-Sent Events

★ PRO TIP — Pick once, stay consistent

Mixing sync and async DB code in the same project is the fastest path to subtle bugs. Pick one style up front. For TaskFlow going forward we use the async stack, because Phase 3 adds WebSockets which require it.

▶ TRY IT YOURSELF — Convert TaskFlow to async

Replace your session module and CRUD functions with the async versions above. Change the router endpoints to `async def` and `await` their CRUD calls. Restart, test in `/docs` — behaviour is identical, the machinery underneath is now fully async.

Alembic Migrations

A live database holds data you must not lose, but your code keeps changing — new columns, renamed fields, new tables. **Alembic** is the migration tool for SQLAlchemy: each schema change becomes a versioned, replayable script.

Why drop `create_all()`?

The `Base.metadata.create_all()` we used in Chapter 4 only creates tables that do not yet exist. It will not add a new column to an existing table, rename anything, or drop unused tables. The day you change a model, your database silently falls out of sync with your code. Migrations fix this by recording every change as a script you can apply or roll back.

A API TERM — Migration

A small, versioned script describing one schema change — 'add a `priority` column to `tasks`'. Migrations chain together: each knows which one came before it. Running them in order brings any database — yours, a teammate's, production — to the current schema.

Install and initialize

```
pip install alembic
# From your project root:
alembic init alembic
```

This creates an `alembic/` folder (where migration scripts live) and an `alembic.ini` config file.

Point Alembic at your database and models

Two small edits make Alembic aware of your project. First, the connection URL in `alembic.ini`:

alembic.ini

```
sqlalchemy.url = sqlite:///./taskflow.db
```

Then, in `alembic/env.py`, import your `Base` and point Alembic's `target_metadata` at it. This is what enables autogenerate.

alembic/env.py

```
# alembic/env.py (only the changed lines shown)
from app.db.base import Base
import app.models.task           # noqa - ensures models register on Base
import app.models.user           # noqa
```

```
target_metadata = Base.metadata
```

NOTE — Import every model module

Alembic's autogenerate compares `Base.metadata` against the live database. If a model module is never imported, its tables are invisible to Alembic. Importing them in `env.py` (or in a central `app/db/base.py`) guarantees they are registered.

Use the sync URL for Alembic

Even when your application runs async, Alembic operations are short and run synchronously. Use the plain sync URL — `sqlite:///...` or `postgresql:///...` — in `alembic.ini`, not the async variant. Less complexity, fewer surprises.

Create your first migration

```
alembic revision --autogenerate -m "create users and tasks tables"
```

This compares your models against the database and writes a script into `alembic/versions/`. Open it — it should declare your tables. Always read every autogenerated script: Alembic occasionally needs nudging (renames, server defaults).

```
# alembic/versions/abc123_create_users_and_tasks_tables.py
def upgrade() -> None:
    op.create_table(
        "users",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("email", sa.String(120), nullable=False, unique=True),
        ...,
    )
    op.create_table(
        "tasks",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("title", sa.String(120), nullable=False),
        sa.Column("owner_id", sa.Integer(),
                  sa.ForeignKey("users.id"), nullable=False),
        ...,
    )

def downgrade() -> None:
    op.drop_table("tasks")
    op.drop_table("users")
```


Apply migrations

```
alembic upgrade head      # apply all pending migrations
alembic current           # show what's applied
alembic history           # see the chain
```

`head` means "the latest." Run this in development after pulling new code, in CI before tests, and as part of every deployment.

Evolving the schema

Real workflow when you change a model:

1. Edit the model (e.g. add `priority: Mapped[int] = mapped_column(default=0)`).
2. `alembic revision --autogenerate -m "add priority to tasks"`
3. Open the generated script. Confirm it does what you expect — and only that.
4. `alembic upgrade head` to apply it locally.
5. Commit the model change *and* the migration file together.

! WATCH OUT — Migrations belong in version control

The migration scripts in `alembic/versions/` are code. They are how production and your teammates reach the same schema as you. Commit them. Never edit a migration that has been deployed — write a new one to fix it.

Rolling back

```
alembic downgrade -1      # back out the last migration
alembic downgrade base    # revert all migrations (development only)
```

Downgrades exist for development and recovery. Treat production downgrades as a careful, rehearsed action — for many real-world changes (column drops, data conversions), forward fixes are safer than rolling back.

★ PRO TIP — Remove `create_all()` once Alembic is in place

Delete `Base.metadata.create_all(bind=engine)` from your lifespan once migrations are real. Two mechanisms managing the same schema is a recipe for confusion — Alembic is now the single source of truth.

▶ TRY IT YOURSELF — Bring your TaskFlow database under Alembic

Run the init, edit the URL and `env.py`, autogenerate the first migration, and apply it. Then add a `priority` field to `Task` and create a second migration. `alembic history` should now show two revisions in a chain.

Authentication — OAuth2 & JWT

Up to now, every endpoint of TaskFlow is open to the world. Time to introduce real users with passwords and protected endpoints. We will follow the official FastAPI tutorial's current approach: **OAuth2 password flow** for login and **JWT** for the access token.

The vocabulary, decoded

A API TERM — OAuth2 / Password flow / JWT / Bearer token

OAuth2 is a flexible authorisation specification. Its **password flow** is the simplest variant: the user posts username and password, the server hands back an access token. A **JWT** (JSON Web Token) is a compact, signed string carrying claims (who the user is, when the token expires). The client then sends it on every request in an `Authorization: Bearer <token>` header — a **bearer token**: whoever holds it is treated as the user, so guard it carefully.

The whole flow in one picture:

- | | |
|-----------|---|
| 1. Client | POST /auth/login { username, password } |
| 2. Server | verify password against the hash stored in the DB
build a JWT containing the user id and an expiry
respond with { "access_token": "<jwt>", "token_type": "bearer" } |
| 3. Client | stores the token, sends it on every protected request:
Authorization: Bearer <jwt> |
| 4. Server | for each request: parse the token, verify the signature
check the expiry, look up the user, inject as a dependency |

Install the modern stack

The current FastAPI documentation uses **PyJWT** for token handling and **pwdlib** for password hashing. (Older tutorials show `python-jose` and `passlib`; both still work, but `pwdlib` is the newer recommendation.)

```
pip install "pyjwt[crypto]" pwdlib[bcrypt] python-multipart
```

Hashing passwords

A API TERM — Password hashing

You never store passwords. You store a one-way **hash** — a deterministic, irreversible scramble. To verify a login attempt, you hash what the user typed and compare it to the stored hash. **bcrypt** is the recommended algorithm: deliberately slow to compute, which makes guessing attacks expensive.

```
app/core/security.py
```

```
# app/core/security.py
from pwdlib import PasswordHash

password_hash = PasswordHash.recommended() # bcrypt under the hood

def hash_password(plain: str) -> str:
    return password_hash.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    return password_hash.verify(plain, hashed)
```

The User model and schema

app/models/user.py

```
# app/models/user.py
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column
from app.db.base import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(120), unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255))
    is_active: Mapped[bool] = mapped_column(default=True)
```

app/schemas/user.py

```
# app/schemas/user.py
from pydantic import BaseModel, EmailStr, Field

class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(min_length=8, max_length=128)

class UserPublic(BaseModel):
    id: int
    email: EmailStr
    is_active: bool

    model_config = {"from_attributes": True}

class Token(BaseModel):
```

```
access_token: str
token_type: str = "bearer"
```

Notice `UserPublic` has no password field at all. Even if you accidentally return a full user from an endpoint, the `response_model` strips the hash before it leaves the server — exactly the safety net Phase 1 Chapter 7 set up.

Issuing JWTs

```
# app/core/security.py (continued)
from datetime import datetime, timedelta, timezone
import jwt

SECRET_KEY = "change-me-in-production"  # load from env in real life
ALGORITHM = "HS256"
TOKEN_EXPIRE_MINUTES = 60

def create_access_token(subject: str | int,
                        expires_minutes: int = TOKEN_EXPIRE_MINUTES) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=expires_minutes)
    payload = {"sub": str(subject), "exp": expire}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
```

! WATCH OUT — Never commit the secret key

The `SECRET_KEY` signs every token. If it leaks, anyone can forge logins. Always load it from an environment variable (we use `pydantic-settings` in Phase 4) and rotate it if you suspect exposure. Don't hardcode it, don't commit it.

The login endpoint

FastAPI ships `OAuth2PasswordRequestForm` — a dependency that reads the standard OAuth2 form fields (`username`, `password`) from the request body. Use it and the Swagger UI gets an *Authorize* button automatically.

`app/routers/auth.py`

```
# app/routers/auth.py
from typing import Annotated
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy import select
from app.db.session import SessionDep
from app.core.security import verify_password, create_access_token
from app.models.user import User
from app.schemas.user import Token

router = APIRouter(prefix="/auth", tags=["Auth"])
```

```

@router.post("/login", response_model=Token)
async def login(
    form: Annotated[OAuth2PasswordRequestForm, Depends()],
    db: SessionDep,
):
    user = (await db.scalars(
        select(User).where(User.email == form.username)
    )).first()
    if user is None or not verify_password(form.password, user.hashed_password):
        raise HTTPException(
            status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return Token(access_token=create_access_token(user.id))

```

The current-user dependency

One small dependency unlocks every protected endpoint: parse the token, verify it, fetch the user. Inject this and your route receives a fully-loaded `User` object.

app/dependencies.py

```

# app/dependencies.py
from typing import Annotated
import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from app.db.session import SessionDep
from app.core.security import SECRET_KEY, ALGORITHM
from app.models.user import User

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

CREDENTIALS_EXC = HTTPException(
    status.HTTP_401_UNAUTHORIZED,
    detail="Could not validate credentials",
    headers={"WWW-Authenticate": "Bearer"},
)

async def get_current_user(
    token: Annotated[str, Depends(oauth2_scheme)],
    db: SessionDep,
) -> User:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id = int(payload["sub"])
    except (jwt.InvalidTokenError, KeyError, ValueError):
        raise CREDENTIALS_EXC
    user = await db.get(User, user_id)
    if user is None or not user.is_active:
        raise CREDENTIALS_EXC

```

```
return user
```

```
CurrentUser = Annotated[User, Depends(get_current_user)]
```

A API TERM — OAuth2PasswordBearer

A built-in FastAPI dependency that reads the `Authorization: Bearer ...` header, returns the raw token string, and wires the Swagger UI's *Authorize* dialog. The `tokenUrl` is just the path of your login endpoint, so the docs know where to log in.

Protecting endpoints

Inject `CurrentUser` into any route that needs the logged-in user. The presence of the dependency alone makes the route protected — no token, no entry.

```
@router.get("/me", response_model=UserPublic)
async def whoami(user: CurrentUser):
    return user

@router.post("/tasks", response_model=TaskPublic, status_code=201)
async def create_task(data: TaskCreate, user: CurrentUser, db: SessionDep):
    return await crud_task.create(db, data, owner_id=user.id)
```

To protect every route in a router, attach the dependency to the `APIRouter` itself (you saw this pattern in Chapter 2):

```
router = APIRouter(prefix="/tasks", tags=["Tasks"],
                    dependencies=[Depends(get_current_user)])
```

Trying the full flow in /docs

1. Register a user via `POST /users`.
2. Click the *Authorize* button at the top of `/docs`; enter the credentials. Swagger calls `/auth/login` for you.
3. Now call `GET /me` or `POST /tasks` — Swagger automatically attaches the token. Without authorising, the same call returns 401.

★ PRO TIP — Authorization is not Authentication

Authentication answers 'who are you?'; **authorization** answers 'are you allowed to do this?'. JWT gives you the first. To do role checks ('only admins can delete') you add a small dependency that reads `user.role` and raises 403. We expand this with OAuth2 scopes in Phase 3.

 TRY IT YOURSELF — Lock down TaskFlow

Add the User model, registration endpoint, login endpoint, and `get_current_user` dependency. Move the task router behind it. Verify that calling `GET /tasks` without a token returns 401, and that after logging in via the Swagger Authorize button it returns your tasks.

Middleware (CORS & Custom)

Some logic should run on *every* request: setting CORS headers so a browser app can call your API, logging how long each request took, attaching a request id. That cross-cutting layer is called **middleware**.

What middleware is

A API TERM — Middleware

A function that wraps every request: it sees the request before your endpoint, can modify it, calls the next layer to actually run the endpoint, then sees the response on the way out and can modify that too. Middleware runs for *all* routes; dependencies run only for the routes that ask for them.

You stack middleware like layers of an onion. A request travels inward through each, the endpoint runs at the centre, then the response travels outward through them in reverse.

CORS: letting browsers call you

A API TERM — CORS (Cross-Origin Resource Sharing)

Browsers refuse to let a page at `example.com` read responses from `api.example.com` unless the API explicitly opts in with CORS headers. Without CORS configured, your fancy backend works perfectly from `curl` but the frontend developers get cryptic errors. It is the single most common 'why doesn't my API work in the browser?' issue.

FastAPI ships a ready-made `CORSMiddleware`. Two lines:

```
app/main.py

from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="TaskFlow API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",          # Vite dev server
        "https://taskflow.example.com",   # production frontend
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```


CORSMiddleware argument	Meaning	Sensible default
<code>allow_origins</code>	Which front-end URLs may call your API	List your real frontends
<code>allow_credentials</code>	Allow cookies/Authorization across origins	<code>True</code> if you need auth from browser
<code>allow_methods</code>	Permitted HTTP methods	<code>["*"]</code> or list explicitly
<code>allow_headers</code>	Permitted custom request headers	<code>["*"]</code>

! WATCH OUT — Never use `allow_origins=['*']` with credentials

The wildcard is shorthand for 'any site can call you.' Combine it with cookies or auth headers and any malicious page in the world could act on behalf of your logged-in users. List the exact origins that should be allowed.

Writing your own middleware

For custom logic, decorate a function with `@app.middleware("http")`. You receive the incoming `Request` and a `call_next` to invoke the rest of the stack; whatever you return is the final response.

```
import time
from fastapi import Request

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)          # run the endpoint
    elapsed = time.perf_counter() - start
    response.headers["X-Process-Time-ms"] = f"{elapsed * 1000:.1f}"
    return response
```

That single middleware now stamps every response with how long it took to build. Two more patterns you'll use constantly:

```
import uuid

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    request.state.request_id = request_id        # available downstream
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response

@app.middleware("http")
async def log_requests(request: Request, call_next):
    response = await call_next(request)
```

```
print(f"{request.method} {request.url.path} -> {response.status_code}")
return response
```

A API TERM — `request.state`

A free-form per-request scratchpad. Anything you set in a middleware (`request.state.user_id = 42`) is readable in downstream middleware, dependencies, and route handlers via the `Request` object — the standard way to pass context through the stack.

Order matters

Middleware is registered in the order you add it, but executed in *reverse* on the way in (last added runs first). The official guidance: add CORS **last** in code, so it effectively runs first — your error responses then carry the right CORS headers and the browser is happy.

```
# Recommended order:
app.add_middleware(LoggingMiddleware)
app.add_middleware(GZipMiddleware, minimum_size=1000)
app.add_middleware(CORSMiddleware, allow_origins=...) # last
```

What middleware is not for

Tempting but wrong uses:

- **Authentication.** Use a dependency (Chapter 7). Middleware does not know which routes are public.
- **Database transactions.** Use a yield dependency (Chapter 1) so each request cleanly opens and closes its session.
- **Per-route logic.** If only some endpoints need it, a dependency is the right tool.

TRY IT YOURSELF — Add timing and CORS to TaskFlow

Add the `X-Process-Time-ms` middleware and configure `CORSMiddleware` with `http://localhost:5173`. Open `/docs`, run any request, and inspect the response headers in the Curl/Response tab — your timing header should be there.

Error Handling & Custom Exception Handlers

A polished API does not just succeed — it fails in predictable, helpful ways. Phase 1 introduced `HTTPException`; here we go beyond it: domain-specific exceptions, consistent error shapes, and overriding FastAPI's defaults when needed.

The three layers of errors FastAPI gives you

Source	Status	When
<code>HTTPException</code>	You choose	Anywhere you spot a known failure
Validation errors	422	Pydantic rejected an input
Unhandled exceptions	500	A bug — your code raised something unexpected

Each layer is customisable. Together they let you guarantee that every error response in your API has the same shape, which makes client code dramatically simpler.

Domain exceptions: name the failure

Inside your CRUD or business logic, raising raw `HTTPException` couples database code to HTTP. Better: define a small domain exception, then map it to HTTP at the edge.

`app/core/exceptions.py`

```
# app/core/exceptions.py
class TaskFlowError(Exception):
    """Base for our domain errors."""

    class NotFound(TaskFlowError):
        def __init__(self, entity: str, key):
            self.entity, self.key = entity, key
            super().__init__(f"{entity} {key} not found")

    class PermissionDenied(TaskFlowError):
        pass
```

```
# CRUD raises domain exceptions, not HTTPException
async def get(db: AsyncSession, task_id: int) -> Task:
    task = await db.get(Task, task_id)
    if task is None:
```

```
raise NotFound("Task", task_id)
return task
```

Mapping domain exceptions to HTTP

Use `@app.exception_handler` at the app level. One handler, one consistent response shape — everywhere your code raises that exception.

```
from fastapi import Request
from fastapi.responses import JSONResponse
from app.core.exceptions import NotFound, PermissionDenied

@app.exception_handler(NotFound)
async def handle_not_found(request: Request, exc: NotFound):
    return JSONResponse(
        status_code=404,
        content={
            "error": "not_found",
            "entity": exc.entity,
            "id": exc.key,
            "request_id": getattr(request.state, "request_id", None),
        },
    )

@app.exception_handler(PermissionDenied)
async def handle_forbidden(request: Request, exc: PermissionDenied):
    return JSONResponse(
        status_code=403,
        content={"error": "forbidden", "detail": str(exc)},
    )
```

? WHY IT MATTERS — Consistency is the goal

Without this layer, half your endpoints return `{{"detail": "..."}}}` and half return `{{"error": "..."}}}` — and frontend code has to handle every variation. With it, every error from your API has the same JSON shape, the same fields, and (with the middleware from Chapter 8) the same request id you can use for debugging.

Customising validation errors

Pydantic's default 422 response is correct but verbose. Override `RequestValidationError` to return your project's standard shape.

```
from fastapi.exceptions import RequestValidationError

@app.exception_handler(RequestValidationError)
async def handle_validation(request: Request, exc: RequestValidationError):
    # Strip noisy fields, keep what's actionable for the client
```

```

fields = [
    {"field": ".".join(str(p) for p in e["loc"][1:]),
     "message": e["msg"], "type": e["type"]}
    for e in exc.errors()
]
return JSONResponse(
    status_code=422,
    content={"error": "validation_error", "fields": fields},
)

```

★ PRO TIP — Keep the original errors

If you wholly replace the validation handler, you lose Pydantic's helpful detail. The common pattern is to transform it (as above) — keep the field path and message, drop the internal stuff. Clients can show 'Title: too short' instead of 'Internal validation tree node x.y.z'.

Overriding HTTPException itself

If you want every error in the API (including the ones you raise as `HTTPException`) to use your custom shape, override its handler too.

```

from fastapi import HTTPException

@app.exception_handler(HTTPException)
async def handle_http_exc(request: Request, exc: HTTPException):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": "http_error", "status": exc.status_code,
                "detail": exc.detail},
        headers=exc.headers,
    )

```

Unhandled exceptions: log, don't leak

A bug should not reveal internal details to clients. Catch `Exception` at the app level, log it (with the request id), and return a generic 500.

```

import logging
log = logging.getLogger("taskflow")

@app.exception_handler(Exception)
async def handle_unexpected(request: Request, exc: Exception):
    rid = getattr(request.state, "request_id", "-")
    log.exception("Unhandled error rid=%s path=%s", rid, request.url.path)
    return JSONResponse(
        status_code=500,
        content={"error": "internal", "request_id": rid},
    )

```

! WATCH OUT — Don't echo the exception text to the client

Stack traces and internal messages can leak file paths, query fragments, even secrets. Return a generic body to the client and log the full detail server-side. Pair this with the request id from Chapter 8 and you can correlate any user-reported failure with the exact log line.

▶ TRY IT YOURSELF — Give TaskFlow a uniform error shape

Define `NotFound` in `app/core/exceptions.py`, convert CRUD to raise it, register the handler. Hit `GET /tasks/9999` and check the response body and status. Then send a bad `POST /tasks` body and see your customised 422.

Background Tasks

Some work doesn't need to finish before the response goes back: sending a "task created" email, writing an audit log, warming a cache. FastAPI's `BackgroundTasks` lets you defer that work cleanly.

What it does

A API TERM — Background tasks

Work scheduled to run *after* the response is sent to the client. The user gets a fast reply; your code finishes the slow part in the background. Same process, same event loop — perfect for short, fire-and-forget jobs.

You declare it like any FastAPI parameter and call `.add_task()`:

```
from fastapi import BackgroundTasks

def write_audit_log(user_id: int, action: str) -> None:
    with open("audit.log", "a") as f:
        f.write(f"{user_id} {action}\n")

@router.post("/tasks", response_model=TaskPublic, status_code=201)
async def create_task(
    data: TaskCreate,
    user: CurrentUser,
    db: SessionDep,
    background: BackgroundTasks,
):
    task = await crud_task.create(db, data, owner_id=user.id)
    background.add_task(write_audit_log, user.id, "task.create")
    return task
```

The response goes out immediately; `write_audit_log` runs after, with the same arguments you passed. You can add as many tasks as you want.

Sync and async tasks both work

FastAPI inspects the callable: `async def` tasks are awaited on the event loop; plain `def` tasks run in the threadpool. Pick whichever fits the work.

```
import httpx

async def send_welcome_email(email: str) -> None:
    async with httpx.AsyncClient() as client:
        await client.post("https://mail-service.local/send",
```

```
json={"to": email, "template": "welcome"})
```

```
@router.post("/users", response_model=UserPublic, status_code=201)
async def register(
    data: UserCreate, db: SessionDep, background: BackgroundTasks,
):
    user = await crud_user.create(db, data)
    background.add_task(send_welcome_email, user.email)
    return user
```

What background tasks are good for

Good fit	Bad fit
Short post-response work (under a few seconds)	Heavy CPU jobs (image processing)
Sending one notification email	Sending thousands — use a queue
Writing a log line / audit entry	Anything that must survive a server restart
Cache invalidation, webhook fire-and-forget	Long-running, scheduled, or critical jobs

? WHY IT MATTERS — They are in-process, not durable

A background task lives in the same Python process as your API. Restart the server and pending tasks vanish. For anything that *must* happen — payments, large emails, scheduled reports — you need a proper task queue (Celery, RQ, Arq). We cover Celery in Phase 3. Background tasks are the simple, lightweight first step.

Tasks inherit dependencies safely

A subtlety: if your background task uses a database session, do not pass the request's session in — it gets closed when the response finishes (remember the `yield` dependency from Chapter 1). Open a fresh one inside the task.

```
from app.db.session import AsyncSessionLocal

async def archive_old_tasks() -> None:
    async with AsyncSessionLocal() as db:          # fresh session
        ...
        await db.commit()

@router.post("/admin/archive")
async def trigger_archive(user: CurrentUser, background: BackgroundTasks):
    background.add_task(archive_old_tasks)
    return {"status": "scheduled"}
```


! WATCH OUT — Errors in background tasks are silent by default

If a background task raises an exception, the client has already received a 200 — there is no way to surface the error to them. Wrap the body of every background task in a try/except and log failures (with your request id). You'll catch problems before they compound.

Background tasks vs Celery — quick mental model

	BackgroundTasks	Celery (Phase 3)
Process	Same as API	Separate worker
Survives restart	No	Yes (broker-backed)
Retries / scheduling	No	Yes
Setup cost	Zero	Broker + workers + monitoring
Right for	Quick, optional work	Critical or heavy work

▶ TRY IT YOURSELF — Add an audit log to TaskFlow

Create a `write_audit_log` function and schedule it from `create_task` and `delete_task`. Hit those endpoints, watch the response come back instantly, then look at `audit.log` — your background task wrote to it after the response was sent.

The Complete TaskFlow API

Every Phase 2 capability assembled into one runnable project. The layout below is the one a small team could ship today: dependency injection throughout, an async database, JWT auth, CORS, custom errors, and background tasks.

Final project layout

```
taskflow/
├─ alembic/                # migrations (Ch. 6)
├─ alembic.ini
├─ .env                    # SECRET_KEY, DATABASE_URL
└─ app/
    ├─ main.py             # app, middleware, exception handlers
    ├─ core/
    │   ├─ config.py       # settings loaded from env
    │   ├─ security.py     # hashing + JWT (Ch. 7)
    │   └─ exceptions.py   # domain errors (Ch. 9)
    ├─ db/
    │   ├─ base.py         # DeclarativeBase
    │   └─ session.py      # async engine + get_db (Ch. 5)
    ├─ models/{{user.py, task.py}} # SQLAlchemy ORM (Ch. 4)
    ├─ schemas/{{user.py, task.py}} # Pydantic (Ch. 4)
    ├─ crud/{{user.py, task.py}}   # DB functions (Ch. 4-5)
    ├─ routers/{{auth.py, users.py, tasks.py}}
    └─ dependencies.py         # CurrentUser, pagination (Ch. 1, 7)
```

app/main.py — the integration point

app/main.py

```
from contextlib import asynccontextmanager
import logging
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
from app.core.exceptions import NotFound, PermissionDenied
from app.routers import auth, users, tasks

log = logging.getLogger("taskflow")

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Migrations are applied via Alembic outside of the app process.
    yield

app = FastAPI(title="TaskFlow API", version="1.0.0", lifespan=lifespan)
```

```

# ---- middleware -----
import time, uuid

@app.middleware("http")
async def add_request_id_and_timing(request: Request, call_next):
    request.state.request_id = request.headers.get(
        "X-Request-ID", str(uuid.uuid4()))
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Request-ID"] = request.state.request_id
    response.headers["X-Process-Time-ms"] = f"{(time.perf_counter()-start)*1000:.1f}"
    return response

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ---- exception handlers -----
def _err(status_code: int, payload: dict, request: Request):
    payload["request_id"] = getattr(request.state, "request_id", None)
    return JSONResponse(status_code=status_code, content=payload)

@app.exception_handler(NotFound)
async def _on_not_found(request, exc):
    return _err(404, {"error": "not_found", "entity": exc.entity,
                     "id": exc.key}, request)

@app.exception_handler(PermissionDenied)
async def _on_forbidden(request, exc):
    return _err(403, {"error": "forbidden", "detail": str(exc)}, request)

@app.exception_handler(RequestValidationError)
async def _on_validation(request, exc):
    fields = [{"field": ".".join(str(p) for p in e["loc"])[1:]),
               "message": e["msg"], "type": e["type"]} for e in exc.errors()]
    return _err(422, {"error": "validation_error", "fields": fields}, request)

@app.exception_handler(Exception)
async def _on_unexpected(request, exc):
    log.exception("Unhandled error rid=%s",
                  getattr(request.state, "request_id", "-"))
    return _err(500, {"error": "internal"}, request)

# ---- routers -----
app.include_router(auth.router)

```

```
app.include_router(users.router)
app.include_router(tasks.router)
```

app/routers/tasks.py — the typical feature router

app/routers/tasks.py

```
from fastapi import APIRouter, BackgroundTasks, status
from app.db.session import SessionDep
from app.dependencies import CurrentUser
from app.schemas.task import TaskCreate, TaskPublic
from app.crud import task as crud_task

router = APIRouter(prefix="/tasks", tags=["Tasks"])

def _audit(user_id: int, action: str, task_id: int) -> None:
    with open("audit.log", "a") as f:
        f.write(f"{user_id} {action} {task_id}\n")

@router.get("", response_model=list[TaskPublic])
async def list_tasks(user: CurrentUser, db: SessionDep,
                    skip: int = 0, limit: int = 20):
    return await crud_task.list_for_owner(db, user.id, skip, limit)

@router.post("", response_model=TaskPublic, status_code=status.HTTP_201_CREATED)
async def create_task(data: TaskCreate, user: CurrentUser, db: SessionDep,
                    background: BackgroundTasks):
    task = await crud_task.create(db, data, owner_id=user.id)
    background.add_task(_audit, user.id, "task.create", task.id)
    return task

@router.get("/{task_id}", response_model=TaskPublic)
async def get_task(task_id: int, user: CurrentUser, db: SessionDep):
    task = await crud_task.get_owned(db, task_id, user.id) # raises NotFound
    return task

@router.delete("/{task_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_task(task_id: int, user: CurrentUser, db: SessionDep,
                    background: BackgroundTasks):
    task = await crud_task.get_owned(db, task_id, user.id)
    await crud_task.delete(db, task)
    background.add_task(_audit, user.id, "task.delete", task_id)
    return None
```

What you can now do

- ✓ Compose endpoints with Dependency Injection — pagination, sessions, current user
- ✓ Organise a growing API into focused routers and a layered project structure
- ✓ Decide correctly between `def` and `async def`

- ✓ Persist data with SQLAlchemy 2.0, both synchronously and asynchronously
- ✓ Evolve a database schema safely with Alembic migrations
- ✓ Register and log in users, issue JWT tokens, protect endpoints
- ✓ Configure CORS for a browser frontend and write your own middleware
- ✓ Return consistent, project-shaped error responses for every kind of failure
- ✓ Defer post-response work with background tasks

Glossary of Intermediate Terms

Every new concept from Phase 2 in one place.

Term	Meaning
Alembic	The migration tool for SQLAlchemy; versioned scripts that evolve the schema safely.
AsyncSession	SQLAlchemy's async equivalent of <code>Session</code> ; methods are awaited.
APIRouter	A grouping object you mount with <code>include_router</code> to split routes across files.
Authentication	Identifying who the caller is (login).
Authorization	Deciding whether the caller is allowed to perform an action.
Background task	Work scheduled to run after the response is sent; in-process and non-durable.
Bearer token	A token that grants access simply by being presented; sent as <code>Authorization: Bearer ...</code> .
bcrypt	A deliberately slow password-hashing algorithm; the recommended choice for storing passwords.
CORS	Browser security rules that require an API to explicitly allow other origins to read its responses.
CORSMiddleware	FastAPI's built-in middleware that sets the headers required by CORS.
CRUD layer	Pure database functions, ignorant of HTTP; lives in <code>app/crud/</code> .
DeclarativeBase	The base class for SQLAlchemy 2.0 models; <code>class Base(DeclarativeBase): pass</code> .
Dependency Injection	Declaring what a function needs and letting the framework provide it via <code>Depends()</code> .
Depends()	FastAPI's marker that turns a callable into a dependency to be resolved per request.
Domain exception	An application-level error class (e.g. <code>NotFound</code>) mapped to HTTP by an exception handler.
Engine	SQLAlchemy's connection-pool object; usually created once per process.
Exception handler	A function registered with <code>@app.exception_handler</code> that converts an exception into a response.
Eager loading	Fetching related rows up front with <code>selectinload</code> / <code>joinedload</code> — essential in async SQLAlchemy.

Term	Meaning
from_attributes	Pydantic config flag that lets a schema build itself from ORM objects (formerly <code>orm_mode</code>).
HS256	An HMAC-SHA256 JWT signing algorithm; uses one shared secret key.
JWT	JSON Web Token — a compact, signed token carrying claims like the user id and an expiry.
Lifespan	An async context manager passed as <code>lifespan=</code> to FastAPI; replaces the old <code>on_event</code> hooks.
Mapped / mapped_column	SQLAlchemy 2.0 type-hint-friendly syntax for declaring model columns.
Middleware	A function that runs on every request and response; for cross-cutting concerns like CORS, logging, timing.
OAuth2	An authorisation standard; FastAPI uses its password flow plus JWT for typical APIs.
OAuth2PasswordBearer	FastAPI dependency that extracts the bearer token and powers Swagger's Authorize button.
OAuth2PasswordRequestForm	FastAPI dependency that reads username/password from the request form for the login endpoint.
ORM	Object-Relational Mapper — lets you treat database rows as Python objects.
PyJWT	The current recommended library for encoding/decoding JWTs in FastAPI projects.
pwdlib	The modern password-hashing library used in current FastAPI tutorials; bcrypt under the hood.
RequestValidationError	The exception FastAPI raises when Pydantic rejects an input; customisable for consistent 422 shape.
Session (sync)	A SQLAlchemy database conversation for one request; opened and closed by a yield dependency.
sub claim	Standard JWT field for the subject — typically the user id.
Sub-dependency	A dependency that itself depends on other dependencies; FastAPI resolves the chain and caches per request.
Yield dependency	A dependency that <code>yield</code> s a value; setup runs before, teardown after the response.

What Comes Next

TaskFlow is now a real, structured, secure, database-backed API. Phase 3 puts it under pressure: real-time features, scale, observability, and proper testing.

COMING IN PHASE 3 · ADVANCED



An observable, real-time, well-tested API

The next phase adds live updates over WebSockets, caching, a proper task queue, rate limiting, structured logs, monitoring, and a real test suite.

Phase 3 topics

- **Advanced Pydantic v2** — validators, computed fields, model serialization tricks
- **WebSockets** — bidirectional, low-latency real-time messaging
- **Lifespan & startup/shutdown events** — deep dive into resource management
- **Rate limiting** with slowapi
- **Caching & Redis** (aioredis) — a fast layer in front of expensive queries
- **Task queues with Celery** — durable, retryable background work
- **Logging & structured logs** — making your API debuggable in production
- **Testing** — pytest, TestClient, mocking dependencies, async tests
- **Monitoring** with Prometheus and Sentry

Before you move on — solidify Phase 2

- ✓ Rebuild TaskFlow from memory; the layout should now feel natural, not memorised
- ✓ Add a `Project` model with a one-to-many relationship to tasks; write the migration
- ✓ Add role-based authorization: only owners can delete their tasks
- ✓ Inspect the auto-generated `/openapi.json` and find every endpoint your auth is protecting
- ✓ Practice a forward-fix migration: add a non-nullable column with a server default, then drop the default in the next migration

i NOTE — You are now a confident FastAPI developer

Dependency Injection, a real database with migrations, JWT auth, middleware, and structured errors are the patterns most production FastAPI codebases revolve around. Phase 3 will feel less like new ideas and more like sharpening the ones you already have.

End of Phase 2. Verified against the official FastAPI documentation at fastapi.tiangolo.com — 2026 edition. FastAPI 0.132+, Python 3.10+, SQLAlchemy 2.0, PyJWT, pwdlib.